Supplementary Material for SmartDyGNN: Dynamic Sparse Training for GNNs via Structural and Gradient Fusion

Abstract

This supplementary material presents comprehensive details omitted from the main manuscript, including in-depth ablation analysis, training dynamics, hyperparameter sensitivity, generalization results on GAT, the complete algorithm pseudocode, as well as extended discussions on limitations and future directions.

I. Detailed Ablation Analysis

This section provides thorough explanations for each ablation variant to clarify the functional roles of different components.

Removing gradient signals (–Grad): The activation ratio surges to approximately 99.7%, accompanied by a minor accuracy drop. This observation confirms that gradient signals serve as an essential bridge between node importance and the learning objective. Without such guidance, the mechanism cannot distinguish task-critical updates from redundant computations, resulting in an overly conservative full-activation policy that defeats the purpose of sparsification.

Removing structural redundancy (–LCC): On Cora, performance remains stable, implying that embedding dynamics and gradient signals alone suffice for effective node selection in small, sparse graphs. Nevertheless, the local clustering coefficient provides a critical topological prior for detecting overlapping neighborhood information. Its value becomes increasingly informative in denser graphs or graphs with strong community structures.

Replacing adaptive selection with fixed activation (–Adaptive): Even with full activation (100% nodes), performance cannot surpass the full model. This result highlights the benefits of adaptive thresholding, which dynamically adjusts resource allocation based on evolving training states, including embedding stability and gradient magnitudes.

Collectively, these results reveal a hierarchical evaluation paradigm:

- Structural cues provide static topological context.
- Dynamic signals capture representation evolution during training.
- Gradient cues anchor decisions to the final learning objective.

The synergy among these three complementary signals enables SmartDyGNN to achieve a favorable balance between efficiency and performance.

II. Supplementary Experimental Analysis

A. Training Dynamics Analysis

We analyze the training trajectory of SmartDyGNN by examining the joint evolution of the average active node ratio and validation accuracy. As illustrated in Fig. 1, all datasets start with a warm-up phase (the first 80 epochs) using full-graph training, during which validation accuracy improves rapidly. After dynamic sparsification is activated, the active ratio gradually decreases and stabilizes within 60%–75%, while validation accuracy exhibits only mild and temporary fluctuations before resuming convergence.

This behavior reveals two key insights:

1) As training proceeds, more nodes either converge (with negligible embedding changes) or become structurally redundant. SmartDyGNN effectively captures such evolution and enables progressive sparsification without sacrificing expressiveness.
2) Dynamic node inactivation does not destabilize optimization. The minor accuracy drop after enabling sparsification arises from temporary distribution shifts induced by subgraph construction, yet the model quickly recovers.

Overall, SmartDyGNN follows an *explore-then-compress* strategy: full-graph exploration ensures reliable early-stage learning, while later adaptive compression focuses computation on high-impact updates, achieving both stability and efficiency.

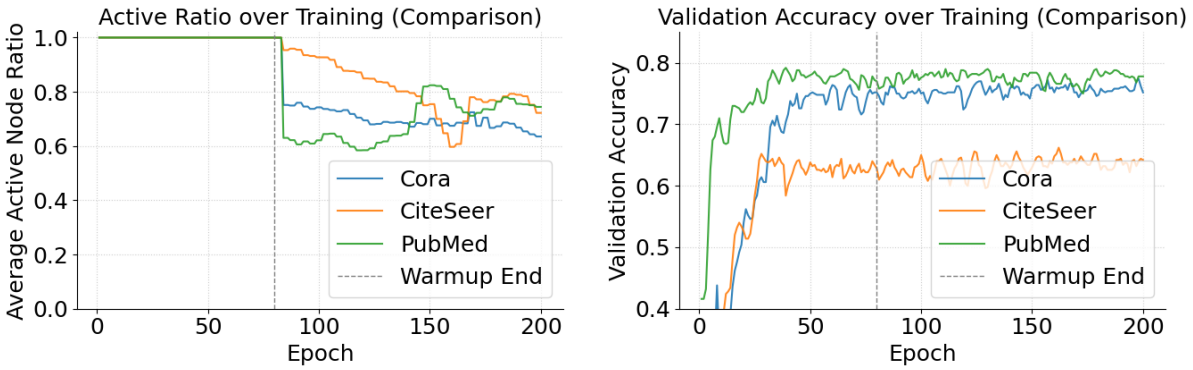

Fig. 1. Training dynamics of SmartDyGNN over epochs. Left: active node ratio. Right: validation accuracy.

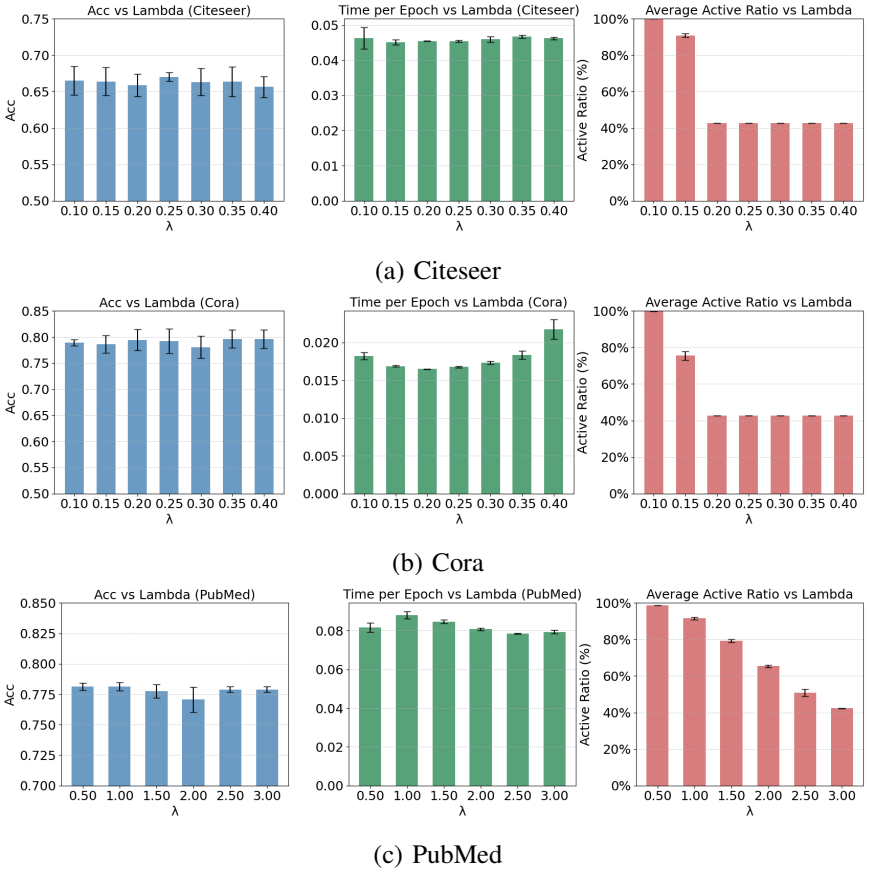

Fig. 2. Hyperparameter sensitivity of λ on three datasets. Each subplot shows test accuracy (left), training time per epoch (middle), and average active node ratio (right).

B. Generalization to GAT

To validate the universality of SmartDyGNN as a training accelerator, we integrate it into GAT to form Smart-GAT. GAT uses learnable attention mechanisms to dynamically weight neighbor contributions, which represents a significantly more complex and input-dependent aggregation strategy than the fixed averaging used in GCN.

As shown in Table I, the resulting performance is nearly identical:

TABLE I
PERFORMANCE OF SMARTGAT ON CITATION NETWORKS

Dataset	Method	Acc. (%)	Active (%)
Cora	GAT	81.42 ± 0.59	100
	SmartGAT	**81.28 ± 0.66**	∼74.2
Citeseer	GAT	72.40 ± 0.74	100
	SmartGAT	**72.58 ± 0.53**	∼70

1) On Cora, SmartGAT achieves 81.28% accuracy, only 0.14% lower than vanilla GAT (81.42%).
2) On Citeseer, SmartGAT slightly improves accuracy by 0.18% (from 72.40% to 72.58%).

Meanwhile, the active node ratio is reduced to roughly 70%.

This result is meaningful because GAT determines *how* to aggregate neighbors, whereas SmartDyGNN determines *whether* a node requires updating. The two mechanisms operate at orthogonal levels of abstraction. Even if a node receives high attention weights from its neighbors, its update can still be safely skipped if its gradient is small or its local structure is redundant, without interfering with attention mechanisms.

Thus, SmartDyGNN acts as a plug-and-play training framework agnostic to the underlying GNN architecture. It can be seamlessly applied to GCN, GAT, GraphSAGE, and other variants, delivering consistent computational reduction with negligible accuracy degradation.

C. Hyperparameter Sensitivity Analysis

We analyze the sensitivity of SmartDyGNN to the hyperparameter λ, which balances the influence of structural redundancy and task-gradient sensitivity in node scoring. As shown in Fig. 2, performance varies across datasets in consistent patterns.

Key observations include:

- Each dataset reflects a distinct accuracy–efficiency trade-off:
  On Cora, peak accuracy (79.2%) and minimal runtime (0.018 s/epoch) both appear at $\lambda = 0.20$. On Citeseer, maximum accuracy (67.1%) appears at $\lambda = 0.25$, and the fastest runtime occurs at $\lambda = 0.20$. On PubMed, accuracy decreases monotonically as λ increases, yet remains robust (78.0% even at $\lambda = 1.0$).
- Dataset-specific behaviors reflect intrinsic graph properties:
  - Cora benefits from balanced weighting, indicating strong sensitivity to task-driven signals.
  - Citeseer, being relatively sparse and noisy, tolerates aggressive pruning.
  - PubMed's compact communities make structural signals highly informative, reducing reliance on gradients.
- The average active ratio decreases smoothly with λ and shows little variance across runs. This stability arises from the *minimum activation guarantee*: if adaptive selection produces too few active nodes, the model falls back to deterministic top-k selection. As a result, the active ratio remains stable across runs, explaining the nearly invisible error bars.

These results motivate future work on automated λ scheduling via meta-learning or graph-property-based adaptation to eliminate manual tuning.

D. Full Performance Comparison

TABLE II
PERFORMANCE COMPARISON WITH STATE-OF-THE-ART METHODS (ACCURACY **MEAN** ± STD).

Method	**Cora**	**Citeseer**	**PubMed**
GCN	0.7895 ± 0.0021	0.6616 ± 0.0032	0.7805 ± 0.0018
DropEdge	0.7876 ± 0.0019	0.6882 ± 0.0025	0.7840 ± 0.0020
ClusterGCN	0.7768 ± 0.0023	0.6713 ± 0.0028	0.7817 ± 0.0017
GraphSAINT	0.7986 ± 0.0015	0.6764 ± 0.0030	0.7561 ± 0.0022
SmartDyGNN	$\mathbf{0.7917 \pm 0.0016}$	$\mathbf{0.6640 \pm 0.0024}$	$\mathbf{0.7847 \pm 0.0014}$

Table II summarizes the performance comparison between SmartDyGNN and representative baselines on all three datasets. Our method achieves highly competitive or superior accuracy while reducing computational costs by 25%–30%, demonstrating its effectiveness and efficiency.

III. Extended Limitations and Future Work

Despite its effectiveness, SmartDyGNN has several limitations:

- The current CPU implementation introduces non-negligible overhead in dynamic subgraph construction. Future work will integrate GPU-native sparse operators (e.g., PyTorch Sparse, cuSPARSE, cuGraph) to accelerate subgraph generation and message passing, significantly improving end-to-end efficiency on large graphs.
- Structural metrics such as LCC still require pre-processing. We plan to explore lightweight online structural estimators to reduce pre-processing cost.
- The framework is currently designed for full-batch training. Extending it to mini-batch and distributed training requires careful cross-partition state synchronization.

Future directions include:

1) Layer-wise sparsification for finer-grained computation control.
2) Extension to dynamic and temporal graphs where structure evolves over time.
3) Validation in realistic industrial scenarios such as heterogeneous graphs and large-scale recommendation systems.
4) Automated λ scheduling via online learning or meta-learning for fully adaptive training.
5) GPU-accelerated sparse operations to fully exploit parallelism on large-scale graphs.

In summary, extensive experiments confirm that SmartDyGNN achieves strong effectiveness, high efficiency, and broad generality across GNN architectures. It reduces computational complexity by 25%–30% while maintaining or improving accuracy, providing a practical and principled approach to resource-efficient graph neural network training.

IV. Complete Algorithm Pseudocode

Algorithm 1 SmartDyGNN: Dynamic Sparse Training

Require: Graph $G = (V, E)$, node features $\mathbf{X}$, warm-up epochs T_{warm}, minimum activation ratio $r_{\min}$, threshold coefficient λ

Ensure: Trained model θ

Precompute node degree d_v and local clustering coefficient c_v
for epoch $t = 1$ **to** T_{max} **do**
  if $t \leq T_{\text{warm}}$ **then** ▷ Warm-up stage
    Standard full-graph forward and backward propagation
  else ▷ Adaptive sparse training
    $\mathbf{X}_{\text{in}} \leftarrow \mathbf{X}$.requires_grad()
    for layer $l = 1$ **to** $L - 1$ **do**
      if $l > 1$ **then**
        $\Delta_v = \|\text{norm}(\mathbf{h}_v^{(l-1)}) - \text{norm}(\mathbf{h}_v^{\text{prev}})\|_2$
        $\tau_v = \lambda \cdot d_v \cdot (c_v + 0.1) \cdot (\bar{g}/g_v)$
        $\mathcal{M} \leftarrow \mathbf{1}[\Delta_v > \tau_v]$
        if $|\mathcal{M}| < r_{\min}|V|$ **then** $\mathcal{M} \leftarrow \text{Top-}k(\Delta)$
        end if
        $\mathbf{A}_{\text{sub}} \leftarrow$ induced subgraph$(\mathcal{M}, \mathbf{A})$
        $\mathbf{h}_{\text{active}}^{(l)} = \text{GCN}(\mathbf{h}^{(l-1)}[\mathcal{M}], \mathbf{A}_{\text{sub}})$
        $\mathbf{h}^{(l)} \leftarrow \mathbf{h}^{\text{prev}}$, $\mathbf{h}^{(l)}[\mathcal{M}] \leftarrow \mathbf{h}_{\text{active}}^{(l)}$
      else
        $\mathbf{h}^{(1)} = \text{GCN}(\mathbf{X}_{\text{in}}, \mathbf{A})$
      end if
      $\mathbf{h}^{(l)} = \text{ReLU}(\mathbf{h}^{(l)})$
    end for
    Compute loss, backpropagate, and update θ
    Save gradient norms $g_v = \|\nabla_{\mathbf{h}^{(l-1)}}[v]\|_2$ for the next epoch
    $\mathbf{h}^{\text{prev}} \leftarrow \mathbf{h}^{(L-1)}$
  end if
end for